

NSF Annual Project Report

Project title: PhyNetPy: An Open-source Library for Developing Phylogenetic Network Inference and Analysis Capabilities

Program: NSF IIBR — Innovation: Bioinformatics (NSF 23-580)

Award number: *[TO BE FILLED IN]*

PI: Luay Nakhleh, Rice University

Reporting period: September 1, 2025 – August 31, 2026 (Year 1)

Report type: Annual

1. Accomplishments

1.1 Major goals of the project

The project develops PhyNetPy, an open-source, general-purpose Python library of data structures, algorithms, and utilities for phylogenetic network inference and analysis. The proposal organizes the work into five thrusts:

1. **Data structures** for phylogenetic networks, trees, and sequence/trait data, supporting flexible models of evolution.
2. **Inference algorithms:** proposal kernels, scoring functions, search templates, priors, and model selection criteria for walking network space, including reimplementations of PhyloNet's inference methods.
3. **Simulation:** generation of synthetic networks, gene genealogies, and sequence/marker data.
4. **Characterization, comparison, and summarization** of networks and sets of networks.
5. **Exploration, diagnostic, and visualization** capabilities.

A secondary deliverable stated in the proposal is a reimplementations of PhyloNet's functionality with improved efficiency and usability.

1.2 What was accomplished under these goals

Summary of scale

At the end of the reporting period the library consists of 40,763 lines of Python across 49 modules, plus four Cython extension modules. The test suite contains 1,090 tests; 1,064 pass and 24 are skipped in 40 seconds on a laptop, with the remainder marked `slow` (long MCMC recovery runs). The package is distributed on PyPI as `phynetpy` and the source is at <https://github.com/NakhlehLab/PhyNetPy>.

Six releases were published to PyPI during the reporting period, taking the library from 0.2.0 to 0.5.0:

Version	Released
0.2.0	2 December 2025
0.3.0	2 March 2026
0.3.1	4 March 2026
0.3.2	2 April 2026
0.3.3	17 April 2026
0.5.0	13 May 2026

All behavioural changes are recorded in `CHANGELOG.md`, with migration tables for each breaking change. (Version 0.4.0 appears in the changelog but was folded into 0.5.0 rather than published separately.) A further set of changes — the unified inference API and the substitution-model audit described below — is complete and staged for the next release.

Thrust 1 — Data structures

Substantially complete.

- **Network representation** (`Network.py`). `Network`, `Node`, `Edge`, and `Branch` implement a rooted directed acyclic graph. Reticulation nodes may have more than two parents, as the proposal required for consensus and uncertainty representation. Nodes and edges carry arbitrary attributes; edges carry lengths, inheritance probabilities (gamma), and weights. Polyploidy is supported through per-node subgenome counts and the associated edge queries, and multiple individuals per species through the species-to-allele mapping carried by every data object. Operations include MRCA, leaf-descendant sets, induced subnetworks, topological ordering, acyclicity checking, topological equivalence testing, copying, and export to NetworkX. No distinction is made between trees and networks; a tree is a network with no reticulation nodes.
- **Input/output** (`IO.py`, `Newick.py`). Four formats are supported for both reading and writing: extended Newick, NEXUS, FASTA, and VCF. The extended Newick reader detects and converts between dialects (`detect_newick_standard`, `convert_newick`), which addresses the lack of a uniform interchange format identified in the proposal as a barrier to the field.
- **Character data** (`Alphabet.py`, `MSA.py`, `Matrix.py`, `data/`). Four alphabets are built in (DNA, RNA, protein, codon) with reverse mappings; biallelic marker data is handled separately. The `data/` subpackage provides `GeneTrees`, `Alignment`, and `BiallelicMarkers`, each of which knows its own type so the user never declares it.
- **Substitution models** (`GTR.py`). Eight models — JC, K80, F81, HKY, TN93, K81, SYM, GTR — with closed-form transition matrices for six of them and cached eigendecomposition for the remaining two. See §4.5 for an unplanned correctness audit of this module. Across-site rate heterogeneity (the "+Γ" of the proposal's Fig. 3(d)) is not yet implemented; the sequence-likelihood engine currently supports a per-locus relative rate but not a discretized gamma distribution over sites. This is scheduled for the next period.

- **Generative process models** (`BirthDeath.py` , `models/`). Yule and constant-rate birth-death processes; the `models/` subpackage exposes `MSC` (multispecies network coalescent) and `Allopolyploid` as first-class, user-selectable models.
- **Data structure efficiency.** Network search evaluates millions of candidate topologies, so the cost of a single edge insertion or deletion, and the per-object memory footprint, set the ceiling on tractable problem size. Both were treated as first-class design constraints rather than later optimizations. `Node` and `Edge` declare `__slots__`, which removes the per-instance dictionary that would otherwise dominate memory in a network with thousands of objects. The node set maintains three indices alongside the nodes themselves — incoming edges by node, outgoing edges by node, and nodes by name — so degree, incidence, and name lookup are constant-time rather than scans over the edge list. The edge set is keyed on the (source, destination) pair, with a list value so that parallel edges arising from reticulation are distinguishable by inheritance probability and tag. Every mutation updates these indices in place instead of rebuilding them: adding or removing an edge touches only the two incident nodes' entries, and renaming a node rehashes only the edges incident to it. On a 999-node, 998-edge network the whole structure occupies 0.96 MB; incidence lookup costs 0.12 μ s; a remove-then-add edge cycle costs 2 μ s; and one thousand such cycles retain 3.46 KB, confirming that repeated topology edits during search do not leak. These figures are asserted as regression tests, not merely measured once: `tests/test_memory.py` holds nine tests that pair timing and allocation bounds with structural invariant checks on in/out-edge bookkeeping, leaf and root classification, and duplicate-edge rejection, so a future change that degrades mutation cost or corrupts the indices fails the suite.
- **Incremental rescoring.** The same concern is carried into the search layer. The pseudo-likelihood scorer distinguishes a topology change, which fully invalidates its cached engine, from a change to branch lengths or inheritance probabilities alone, which does not; a dirty-node set is propagated so that parameter-only proposals reuse the cached decomposition. A differential test (`tests/test_mpl_incremental.py`) checks the incremental scorer against a full rebuild.
- **Compiled kernels.** Four Cython extension modules are compiled at install time, unconditionally. `graph_core_cy` supplies the node and edge set containers described above. `gt_msc_cy` and `mpl_engine_cy` accelerate the gene-tree coalescent likelihood and the pseudo-likelihood dynamic program. `seq_engine_cy` implements the sequence-likelihood branch kernels; it is compiled but not yet wired into `_seq_likelihood.py`, and connecting it is scheduled for the next period.
- **Retirement of the duplicate pure-Python graph core.** The library originally shipped a pure-Python `NodeSet` / `EdgeSet` alongside the compiled one, selected by a fallback import. Maintaining two implementations of the same structure by hand proved unsound: the two had already drifted apart on four behaviors, meaning results could depend on whether the extension had built. The pure-Python duplicate and an orphaned facade module (`graph_core.py`) were removed, and the compiled extension is now a hard requirement that raises an actionable `ImportError` naming the likely cause when it is missing. The two remaining Python paths were kept deliberately and are not vestigial fallbacks: the pseudo-likelihood one serves as the reference implementation for differential testing, and the coalescent one is required for correctness, because the compiled dynamic program packs lineage configurations into a 64-bit mask and is therefore valid only for gene trees of at most 62 nodes, above which the Python path is the only correct route. The optional `[fast]` install extra and the fallback language in `README.md` predate this change and are stale; correcting them is a documentation task for the next period.

Thrust 2 — Inference algorithms

Substantially complete; the primary focus of the reporting period.

- **Unified inference API.** Inference is expressed as two verbs, `infer` and `score`, parameterized on three independent axes: the **data** (`GeneTrees`, `Alignment`, `BiallelicMarkers`), the **model** (`MSC`, `Allopolyploid`), and the **criterion** (`MDC`, `Likelihood`, `PseudoLikelihood`, `Bayesian`). Choosing a method is choosing arguments rather than learning a new command name. This replaced seven separate entry points and five incompatible return types with one `InferenceResult`.
- **Dispatch as an executable validity matrix** (`_registry.py`, `_engines.py`). Whether a run is legal depends on the whole (data, model, criterion) triple. The registry distinguishes three failure modes that were previously conflated: `TypeError` when a criterion is undefined on a data type, `ValueError` when the data lacks required branch lengths, and `NotImplementedError` when a combination is meaningful but not yet built. `validity_matrix()` prints the table and `register()` lets a developer add a cell — this is the extension point for contributed methods. **PhyloNet methods reimplemented.** Six of the eight inference methods the proposal named are implemented and tested.

Proposed method	Status
Bayesian from sequence alignments (Wen & Nakhleh 2018)	<code>MCMC_SEQ</code> — implemented
Bayesian from gene trees (Wen et al.)	<code>MCMC_GT</code> — implemented
Bayesian from biallelic markers (Zhu et al. 2018)	<code>MCMC_BiMarkers</code> — implemented
ML from gene trees (Yu et al. 2014)	<code>InferNetwork_ML</code> — implemented
Maximum pseudo-likelihood from gene trees (Yu & Nakhleh 2015)	<code>InferNetwork_MPL</code> — implemented
Maximum parsimony under polyploidy (Hejase et al.)	<code>MP_Allop</code> — implemented, with bootstrap
Maximum pseudo-likelihood from biallelic markers	Registered, not implemented
Maximum parsimony from gene trees, diploid hybridization	Registered, not implemented

The two unimplemented cells raise `NotImplementedError` rather than failing obscurely, and are visible in `validity_matrix()`. One further limitation is recorded the same way: maximum-likelihood inference from biallelic markers (`MLE_BiMarkers`) can currently score a given network but cannot search, so its `infer` path raises rather than returning a partial result.

Remaining components of this thrust:

- **Proposal kernels.** The proposal committed to providing "over 15 different proposal kernels for traversing the network space." Twenty-two distinct proposal operators are implemented. `ModelMove.py` provides ten reusable move classes — `SPR`, `ChangeNodeHeight`, `ChangeInheritanceProb`, `AddReticulation`, `RemoveReticulation`, `FlipReticulation`, `ChangeReticSource`, `ChangeReticDest`, and `RelocateReticulation`, shared by the gene-tree, marker, and maximum-likelihood searches, plus `SwitchParentage` used by the allopolyploid search. `_mcmc_seq.py` adds twelve operators specific to joint species-network and gene-tree estimation, covering gene-tree node heights and NNI, network node heights, inheritance probabilities, the population-size parameter, and coupled and decoupled variants of reticulation addition, deletion, and relocation. The reversible-jump Hastings ratios for the dimension-changing moves are factored into one

module (`_network_moves.py`) rather than restated per sampler; the add and delete ratios are constructed as exact negatives of one another, and a regression test asserts that an addition followed by the matching deletion has a combined log-Hastings ratio of zero.

- **Proposal-kernel bug fixes.** Three moves (`FlipReticulation`, `ChangeReticDest`, `AddReticulation`) had latent inheritance-probability invariant violations that produced historical acceptance rates of 0%, meaning those regions of network space were unreachable. All three were diagnosed and fixed during the period.
- **Search drivers** (`MetropolisHastings.py`). Hill climbing and simulated annealing with geometric and linear cooling schedules, plateau detection, and adaptive temperature reheating, over a `ProposalKernel` interface that a developer can implement. Every sampler can also be run as a maximizer, so the same kernels serve Bayesian and point-estimate use.
- **Metropolis-coupled MCMC.** `MCMC_SEQ.search(temperatures=[...], swap_interval=...)` runs a tempered ensemble, and `run_parallel_chains` runs independent chains across processes while reporting a live Gelman-Rubin statistic. `MCMC_GT` instead parallelizes likelihood evaluation across gene trees. Both address the poor mixing the proposal anticipated for trans-dimensional network samplers.
- **Adaptive proposals.** `MCMCGTKernel` tunes proposal widths by Robbins-Monro updates, scales move weights by observed acceptance rate, adapts SPR distance decay, reports per-move acceptance statistics and burn-in/tune/sample phase, and freezes adaptation at the end of burn-in to preserve the chain's stationary distribution.
- **Trans-dimensional sampling.** The reticulation add/delete operators in `MCMC_SEQ` implement the dimension-changing moves the proposal identified as necessary, with geometric placement proposals and coupled gene-tree re-proposal to address the poor mixing the proposal anticipated.
- **Search configuration** (`_search_flags.py`). Four presets — `default`, `fast`, `accurate`, and `phylonet` — bundle coherent sets of behaviour flags so users need not combine booleans by hand. The `phylonet` preset exists specifically to reproduce PhyloNet's behaviour for cross-checking. Individual flags (`opt_bl`, `fix_st`, `max_lvl`, `pseudo`, `optimize_scope`) override the preset.
- **Continuous-parameter optimization** (`_optimize.py`). A scorer-agnostic Brent coordinate-ascent optimizer for branch lengths and inheritance probabilities, shared by all three gene-tree methods. Enabling per-topology gamma optimization closed a systematic accuracy gap for networks with one or more reticulations at essentially no runtime cost, and is now the default.
- **Caching.** The proposal's "only do a calculation once" principle is realized in the incremental MPL scorer, which reuses the cached dynamic-programming engine and extracted topology when only continuous parameters change, and is validated bit-for-bit against full rebuilds.
- **Model selection** (`ModelSelection.py`). `reticulation_sweep` fits a range of reticulation counts and reports information criteria, addressing the proposal's concern that more complex networks almost always fit data better.
- **Constrained search.** `Start(net, StartMode.AUGMENT)` requires the result to contain a user-supplied network, enforced by a cluster-containment validator in the accept path rather than approximated by dropping moves.

Thrust 3 — Simulation

Partially complete.

- A third verb, `simulate(model, network, n, data=...)`, runs the inference axes backwards and returns a data-axis object, so a recovery or null-model check composes with an `infer` call and no glue code.
- **Networks:** Yule and constant-rate birth-death processes for species trees (`BirthDeath.py`); networks by drawing a Yule tree and grafting reticulations onto it (`SNPSimulator.random_network`). If no network is supplied, `simulate` draws a species tree itself.
- **Gene genealogies:** backward-in-time coalescent simulation within network branches under the multispecies network coalescent, with lineages routed at reticulations according to inheritance probabilities (`_sim_seq.simulate_gene_tree`).
- **Sequences:** CTMC sequence evolution down each gene tree under JC69, HKY85, or GTR, single-locus and multilocus (`_sim_seq.simulate_sequences`, `simulate_multilocus`).
- **Biallelic markers:** site-by-site two-state CTMC simulation along network branches (`SNPSimulator.simulate`).

Not yet implemented: a first-class birth-death-hybridization process in which hybrid speciation and extinction occur jointly with speciation (SiPhyNetwork-style) — the current network simulator grafts reticulations onto a tree drawn separately, which does not generate networks under a single coherent process; the coalescent serial-founder model with migration for admixture graphs; DLCoal/SimPhy integration; and interfaces to `ms`, `seq-gen`, and `INDELible`. See §4.3. Simulation under the `Allopolyploid` model raises `NotImplementedError`; only `MSC` is supported.

Thrust 4 — Characterization, comparison, and summarization

Partially complete.

- **Constituent components** (`GraphUtils.py`). Displayed trees with their probabilities, the dominant (major) tree, hardwired and softwired clusters, tripartitions, biconnected components (blobs) and the tree of blobs, network level, bridges and articulation points, induced subnetworks by taxon set, and displayed-tree counting. Multi-labelled (MUL) tree conversion is implemented and is the representation the allopolyploid parsimony method operates on.
- **Dissimilarity measures.** Nine network comparison measures are implemented and tested: mu (path-multiplicity) distance, hardwired cluster distance, softwired cluster distance, Robinson-Foulds, tripartition distance, displayed-tree distance, average path distance, weighted average path distance, and rooted triplet distance. Cluster, tripartition, and displayed-tree measures accept a `normalize` argument returning a value in `[0, 1]`.
- **Reticulation-specific comparison** (`ReticulationComparison.py`). A separate module compares networks by the reticulation events they assert rather than by overall topology: reticulation tripartitions, a block-distance construction, reticulation dissimilarity, precision/recall against a reference network, and a combined measure. This supports the common question of whether an inferred network recovers the *right hybridizations*, which whole-network distances answer only indirectly.
- **Testing.** 153 tests cover this thrust: 83 for network distances, 40 for reticulation comparison, and 30 for blobs and subnetworks.

Not yet implemented: a network classification tool covering the graph-theoretic classes (tree-child, tree-based, galled, normal, and others) to replace the outdated NetTest web server — only network level is currently computed; and summarization of *sets* of networks (backbone networks, tree decomposition, hybrid frequencies, consensus networks). See §4.6 for a research note on the latter.

Thrust 5 — Exploration, diagnostics, and visualization

Partially complete.

- **Convergence diagnostics** (`_chain_analysis.py`). Effective sample size, autocorrelation time, standard error of the mean, highest-posterior-density intervals, the Geweke statistic, and the Gelman-Rubin R -hat across chains, with per-parameter and whole-chain summary objects. This covers the ESS and AWTY-style diagnostics the proposal called for.
- **Interoperability of diagnostic output.** Chain traces are written in Tracer log format and tree samples in NEXUS, so existing community tooling can read PhyNetPy output directly. Reading Tracer logs back in is also supported.
- **Hypothesis exploration.** `score(network, data, ...)` evaluates any user-supplied network under any criterion, with `optimize=True` re-optimizing its continuous parameters. This is the proposal's requirement to compare a modified network against a maximum-likelihood estimate without rerunning inference from scratch. `reticulation_sweep` supports the related question of how many reticulations the data supports.
- **Input diagnosis** (`Validation.py`, `Guides/VALIDATION_GUIDE.md`). Validation of input files and network topologies with structured error reporting.
- **Rendering.** `GraphUtils.ascii` renders a network as text, optionally annotated with edge lengths and inheritance probabilities, for terminal inspection and logging.

Not yet implemented: DensiNetwork, integration with IcyTree and Dendroscope, and the graphical user interface and cloud deployment. See §4.4.

Cross-verification against PhyloNet

Because the proposal commits to reimplementing PhyloNet's methods, a harness was built to check numerical agreement against PhyloNet itself rather than against PhyNetPy's own expectations. `tests/crosscheck/` contains a compiled Java driver that invokes PhyloNet's own classes (`GeneTreeBrSpeciesNetDistribution`, and BEAGLE for the phylogenetic likelihood) and a Python runner that feeds identical inputs to both implementations from one shared specification file, so any disagreement is a real discrepancy rather than a difference in test setup. Both the multispecies-network-coalescent branch-length density and the Felsenstein likelihood are compared.

The 14 cases are deliberately adversarial: multiple alleles per species, GTR with skewed base frequencies and asymmetric exchange rates, stacked reticulations, boundary population sizes, embeddings of probability zero (both sides must agree on negative infinity), IUPAC ambiguity codes and gaps, branch-length saturation, fully constant alignments, and a larger five-taxon caterpillar. The suite skips automatically unless a PhyloNet jar and BEAGLE are present, so it does not burden ordinary contributors.

1.3 Key outcomes

1. A working, installable, tested library covering all five thrusts to differing depths, with the inference thrust essentially complete for the methods named in the proposal, released six times to PyPI during the period.
 2. An extension mechanism for the developer community — the dispatch registry — that is exercised by the library's own methods rather than being a speculative abstraction (see §4.1).
 3. Direct numerical cross-validation of the sequence-based likelihood against PhyloNet, which is the evidence that the reimplementations deliverable is met rather than merely claimed.
 4. Correctness work with consequences beyond this library: an audit of the substitution models (§4.5) and the diagnosis and repair of three proposal moves whose historical acceptance rate was zero.
 5. Removal of roughly 3,000 lines of unreachable or unused Python and 109,397 lines of generated C that was never compiled, reducing the surface a new contributor must read.
-

2. Products

Software. PhyNetPy v0.5.0, MIT licensed.

- PyPI: `pip install phynetpy`. Python 3.9–3.13.
- Source: <https://github.com/NakhlehLab/PhyNetPy>
- Dependencies: NumPy, SciPy, scikit-learn, matplotlib, NetworkX, Biopython, python-nexus, newick, PuLP. Four Cython extension modules are compiled at install time and are required; a C compiler is therefore a build prerequisite.

Download statistics for the reporting period are not yet compiled and will be reported next period.

Documentation.

- Generated API reference: 24 module pages plus an index, in `docs/api/`, produced by `generate_docs.py`, an AST-based generator that reads module docstrings and prunes pages for deleted modules.
- Project site: six pages in `docs/` — landing page, documentation index, demos, releases, news, and a community board — publishable through GitHub Pages.
- Guides: installation, input/output, validation, and a style guide, in `Guides/`.
- `CHANGELOG.md` documents every behavioural change, including migration tables for each breaking change.
- Lab website: <https://phylogenomics.rice.edu>. A dedicated project website with tutorials, as committed in the proposal, is not yet stood up.

Runnable examples. Twelve end-to-end scripts in `examples/`, including a quickstart, I/O usage, pseudo-likelihood scoring and search at 7 and 20 taxa, Bayesian gene-tree search, simulation-and-recovery, tree of blobs, search-flag comparison, and an analysis of a yeast dataset. The scoring and search-flag examples complete in seconds.

Benchmark and research tooling. Seventeen scripts in `scripts/` for benchmarking, profiling, and scenario exploration, plus the PhyloNet cross-validation harness in `tests/crosscheck/`. These are not part of the shipped library.

3. Impact

On the principal discipline. The library gives method developers in phylogenetics a Python foundation for network methods where previously only tree-based libraries (DendroPy, ETE, TreeSwift, phangorn, Biopython) existed. The three-axis API means that a developer contributing a new scoring criterion or generative model registers one cell rather than writing a new command, and inherits the existing search drivers, proposal kernels, parameter optimizer, diagnostics, and I/O. The `phylonet` search preset lets results be cross-checked against the established tool, which lowers the barrier to trusting a reimplementation.

On unifying phylogenetics and population genetics. The proposal identified the artificial separation between the phylogenetic-network and admixture-graph communities as an obstacle. Concrete steps this period: VCF read/write support (§4.7), and biallelic-marker data as a first-class data axis on equal footing with gene trees and alignments.

Technology transfer. Distribution through PyPI under the MIT license, with versioned releases and documented migration paths for breaking changes.

4. Changes and problems

This section records departures from the proposed plan, with justification, and work undertaken that the proposal did not anticipate.

4.1 The probabilistic graphical model framework was removed as a runtime abstraction

Proposed: the library would be "based on the powerful probabilistic graphical model (PGM) framework, adapted to phylogenetics following the proposed models of Höhna et al.," with model construction expressed as composition of PGM nodes.

What happened: the PGM scaffolding was built, then removed. In practice it advertised an extension surface nobody could use. `ModelFactory` resolved component dependencies through a priority queue in order to order three components whose order was hardcoded at both call sites. The `Visitor` and `Strategy` abstractions had exactly one subclass each, both private to the biallelic-marker likelihood. The `ModelNode` graph was rebuilt on every likelihood evaluation and was never the same object as the model the search loop mutates, so nothing outside that one algorithm could meaningfully build or walk one.

Justification: the underlying goal — modular, mix-and-match model specification with details abstracted away from the user — is met, and met better, by the three-axis (data, model, criterion) API and its dispatch registry. That design achieves the same separation of the generative process from the optimality criterion from the data type, but every axis is a real user-facing argument and the registry is exercised by all six

implemented methods. A developer extends the library by registering a cell, which is a smaller and better-tested interface than subclassing a component hierarchy. PGMs remain valuable as the *documentation and communication* device Höhna et al. advocate, and the proposal's commitment to publishing the graphical model underlying each feature on the project website is unaffected; the change is that the model diagram is not also the runtime object graph.

4.2 Cython replaced Python.NET/C# for performance

Proposed: "we will address the issue by supplementing the code with Python.NET, which allows C# code to be used and compiled into Python via dynamic link libraries."

What happened: the four accelerated kernels are written in Cython.

Justification: Cython requires no .NET runtime on the user's machine, builds as standard binary wheels installable with `pip`, and is the normal choice in the scientific Python ecosystem the library already depends on (NumPy, SciPy). A .NET dependency would have worked against the proposal's own goal of a library that is "simple, easy to download, and quick to get started." The performance objective is met: the accelerated MPL and MSC kernels are what make the 20-taxon searches in `examples/` tractable. The cost of this choice is that a C compiler is now a build prerequisite, since the graph core is compiled-only; see Thrust 1 in §1.2 for why the parallel pure-Python graph implementation was retired rather than maintained.

4.3 Simulation is native rather than wrapping external tools

Proposed: simulation of sequences by "implementing interfaces to tools such as seq-gen, ms, and INDELible," and gene-family simulation by "automating the generation of such data using the tool ms" and "integrating PhyNetPy with SimPhy."

What happened: coalescent gene-tree simulation, CTMC sequence simulation, and SNP simulation are implemented natively in Python.

Justification: the proposal's stated motivation for wrapping `ms` was that it "requires tedious command-line specification of population splits and mergers." Implementing the coalescent directly removes the tedium and the external binary at once: the user does not have to install, locate, or version-match a separate program, and simulated output is returned as a `GeneTrees` or `Alignment` object that feeds straight into `infer`. This is what makes `simulate` compose with `infer` as a single expression. The trade-off is that PhyNetPy does not yet inherit features unique to those tools — notably indel simulation from INDELible and recombination from `ms`. Interfaces to external simulators remain planned for interoperability rather than as the primary path, and the birth-death-hybridization and serial-founder network models are still to be built.

4.4 The graphical interface, visualization, and cloud deployment are deferred

Proposed: a containerized Blazor/.NET MAUI web and desktop application deployed via Kubernetes and Docker; a DensiNetwork visualization tool; and integration with IcyTree and Dendroscope.

What happened: none of these were started. Network rendering is currently ASCII text.

Justification: effort was directed at the computational core first, on the reasoning that a user interface over methods that are not yet correct or fast is of limited value, and that the interface's design should follow from

a stable API rather than precede it. The API stabilized only late in this period, with the `infer / score / simulate` consolidation. Two considerations partly reduce the urgency: PyPI distribution addresses much of the installation burden that motivated the web application, and Tracer-format and NEXUS output (§Thrust 5) lets users reach existing visualization and diagnostic tools today. These items are scheduled for the next reporting period (§5).

4.5 Unplanned: correctness audit of the substitution models

Not anticipated in the proposal. Every formula in `GTR.py` was checked against its primary source, and the module was found to be substantially incorrect: the rate matrix was not time-reversible for non-uniform base frequencies, because exchangeabilities were mapped to the wrong matrix cells and the stationary frequency factor was omitted; $e^{\{Qt\}}$ was computed with a symmetric eigensolver on a non-symmetric matrix, producing transition matrices whose rows summed to values such as 1.97; K80's closed form returned a constant matrix; and three of the eight models could not be constructed at all.

All are fixed. Every model is now verified against `scipy.linalg.expm` to approximately $1e-14$ by 339 new tests, along with the Chapman-Kolmogorov identity, convergence to the stationary distribution, and the model-nesting relationships. Six models gained exact closed forms, 2.9x to 5.4x faster than a matrix exponential.

Justification for reporting this as progress: substitution models are the building block on which the sequence-based methods in Thrust 2 and the sequence simulation in Thrust 3 rest, and a library whose stated purpose is to let others build on it cannot ship incorrect ones. No published inference result changes: nothing on the inference path used these classes, since the sequence-likelihood engine has its own independent implementation. The two implementations now agree exactly, which cross-validates both.

4.6 Unplanned: negative result on consensus networks

A written research note (`src/Theory/network_consensus_geometry.md`) records a literature review of summarizing a sample of networks into a consensus network with support values — a Thrust 4 deliverable. The review found that most of what had been drafted as a contribution was either definitional, long established, or already published between 2019 and 2024, leaving a narrow and uncertain remaining gap. The note is retained, explicitly claiming no result, so that the reasoning is not repeated. Set-level summarization is consequently the least developed part of Thrust 4.

4.7 Unplanned: VCF support

The proposal named extended Newick, NEXUS, FASTA, and PHYLIP. VCF read/write was added instead of PHYLIP because VCF is the standard format for the variant data used by the population-genetics community that the proposal aims to reach, and biallelic markers are a supported data axis. PHYLIP remains to be added.

4.8 No continuous integration yet

The test suite is comprehensive and fast (1,090 tests in 40 seconds) but runs only on developer machines; there is no `.github/workflows/` configuration, so tests are not run automatically on push or pull request and release wheels are built on one machine. This is a gap for a project whose purpose is to accept outside contributions, and it is the first item scheduled for the next period (§5). It was not addressed sooner because

the module layout and public API were still moving; pinning a build matrix to a structure that was about to change would have been wasted effort.

4.9 Public API breakage

Consolidating seven inference entry points into `infer` and `score` is a breaking change, as is the substitution-model reparameterization (§4.5). Both are documented with complete migration tables in `CHANGELOG.md`, and every old call either raises an error or is a mechanical rewrite; none silently changes meaning. The library is at version 0.x and classified "Development Status :: 3 - Alpha", and the judgment was that it is better to settle the interface before an external user base accumulates than after.

5. Plans for the next reporting period

Thrust 2. Implement the two registered but unbuilt cells: maximum pseudo-likelihood from biallelic markers, and maximum parsimony from gene trees under diploid hybridization. Extend the pseudo-likelihood approach to the biallelic-marker and sequence data axes, which requires scorers that decompose over taxon subsets. Implement the sub-network "gluing" facility for divide-and-conquer inference described in the proposal.

Thrust 3. Add a birth-death-hybridization network model with hybrid speciation and extinction, and the coalescent serial-founder model with migration for admixture graphs. Add interfaces to external simulators for interoperability. Add gene duplication and loss to the gene-evolution model.

Thrust 4. Build the network classification tool covering the current graph-theoretic classes, with a registration mechanism so the community can add classes. Implement set-level summarization: backbone networks, tree decomposition, major trees, and hybrid frequencies.

Thrust 5. Begin visualization: DensiNetwork, and export paths to IcyTree and Dendroscope. Design the user interface against the now-stable API, and evaluate the deployment stack — the .NET-based plan in the proposal warrants revisiting given the decision in §4.2.

Cross-cutting. Add continuous integration and per-release binary wheels across the supported Python versions (§4.8). Stand up the dedicated project website with tutorials and a discussion forum, and begin the video walkthroughs committed in the proposal. Add PHYLIP I/O. Begin collecting download and adoption statistics. Assemble the benchmark dataset collection, both empirical and simulated, that the proposal commits to distributing with the library.